

R4.ROM.09

Outils DevOps

Configuration d'une infrastructure réseau *à l'aide d'Ansible*

Compte-rendu de TD/TP

Ponthieux Léo

Année universitaire 2025-2026

Sommaire

01	Introduction	3
02	Mise en place de l'environnement	4
03	Inventaire et lecture de la configuration	6
04	Configuration des interfaces du routeur	14
05	Serveur DHCP	18
06	NAT et PAT dynamique	22
07	PAT par port (redirections)	23
08	Déploiement du serveur web	24
09	Conclusion	26

01

Introduction

Ce compte-rendu présente la réalisation du TD/TP du module R4.ROM.09 (Outils DevOps), dont l'objectif est de mettre en œuvre l'outil Ansible pour automatiser la configuration d'une infrastructure réseau d'entreprise. Le projet consiste à provisionner, à partir d'une VM dédiée au déploiement, un routeur Cisco CSR1000v ainsi qu'un serveur web Debian, en s'appuyant exclusivement sur des fichiers de description au format YAML.

L'infrastructure simulée comporte trois machines virtuelles. Le routeur CSR1000v assure le rôle de passerelle entre Internet et trois réseaux internes : LAN_ENTREPRISE, LAN_DMZ et LAN_ADMIN. Un serveur web Debian, hébergé en DMZ, doit être joignable depuis l'extérieur via une redirection de port. La VM Ansible, placée dans le réseau d'administration, exécute les playbooks et reste accessible depuis l'extérieur grâce à une seconde redirection de port vers son service SSH.

Récapitulatif de l'adressage

Élément	Réseau / Interface	Adresse IP
Routeur CSR1000v - G1 (WAN)	Bridge filaire	DHCP
Routeur CSR1000v - G2	LAN_ENTREPRISE	192.168.2.1/24
Routeur CSR1000v - G3	LAN_DMZ	192.168.3.1/24
Routeur CSR1000v - G4	LAN_ADMIN	192.168.4.1/24
Serveur web Apache	LAN_DMZ	192.168.3.2/24
VM Ansible (déploiement)	LAN_ADMIN	192.168.4.2/24

Deux redirections de port sont mises en place sur l'interface WAN du routeur : le port 80 est dirigé vers le serveur web, et le port 2222 vers le SSH de la VM Ansible. Cette dernière redirection nous permettra de continuer à piloter l'infrastructure depuis l'extérieur après avoir basculé la VM Ansible dans le réseau d'administration.

Service exposé	Endpoint externe	Cible interne
Site web (HTTP)	IP_WAN_CSR : 80	192.168.3.2 : 80
SSH Ansible	IP_WAN_CSR : 2222	192.168.4.2 : 22

02

Mise en place de l'environnement

Configuration des machines virtuelles

Les deux machines virtuelles fournies (CSR1000vR409.ova et Ansible_etudiant.ova) ont été importées dans VirtualBox.

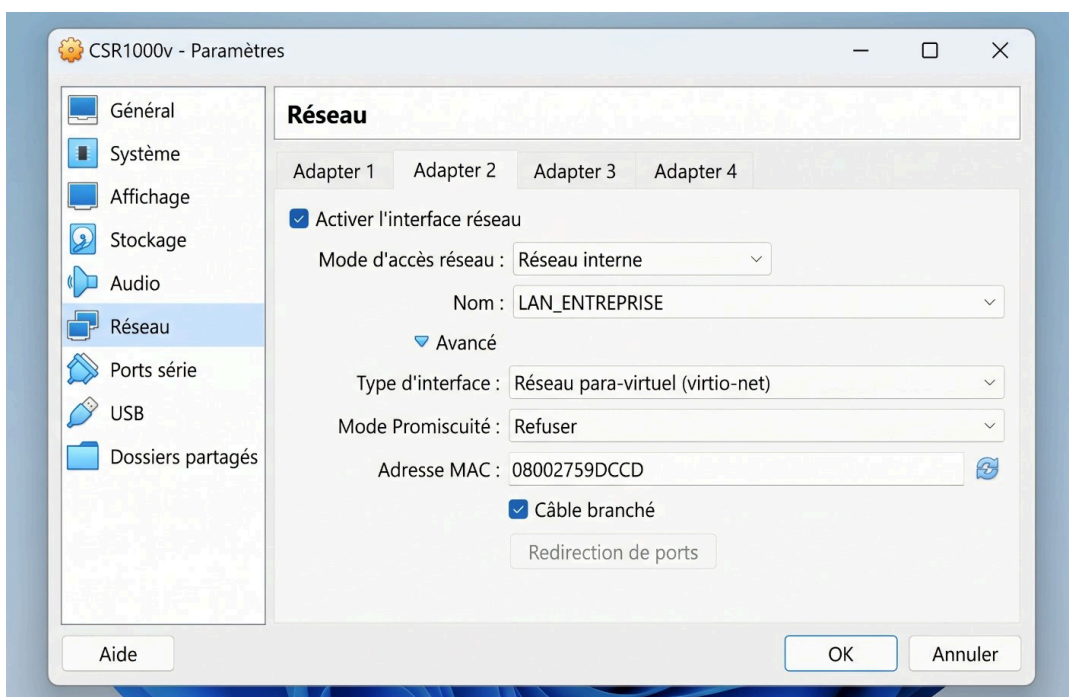


Figure 1 – Configuration de l'interface Adapter 2 du CSR1000v dans VirtualBox

La capture ci-dessus illustre la configuration de l'Adapter 2 du routeur, raccordé au réseau interne LAN_ENTREPRISE. Les trois autres interfaces ont été configurées de manière analogue (Adapter 1 en pont sur l'interface filaire, Adapter 3 sur LAN_DMZ, Adapter 4 sur LAN_ADMIN), toutes en virtio-net avec le câble branché.

Accès SSH depuis le poste de travail

Pour faciliter l'édition des fichiers et la prise de notes, l'ensemble des manipulations est réalisé depuis un PC portable distant, via SSH vers la VM Ansible. Une paire de clés Ed25519 a été générée puis déposée sur la VM avec `ssh-copy-id`, ce qui évite de saisir le mot de passe à chaque connexion. Un alias est configuré dans `~/.ssh/config` :

```
Host ansible-vm
  HostName 192.168.1.51
  Port 2222
  User utilisateur
  ForwardAgent yes
```

L'option `ForwardAgent` permet à la VM Ansible d'utiliser ma clé privée pour rebondir vers le serveur web sans qu'une copie de la clé soit présente sur la machine de déploiement.

Création du dépôt Git

Un dépôt public `td_ansible_infra` a été créé sur GitHub afin de versionner l'ensemble des fichiers du projet. La configuration locale s'effectue avec `git config`, puis le dépôt est lié à l'origine distante via HTTPS et un Personal Access Token (le mot de passe historique n'étant plus accepté par GitHub depuis 2021).

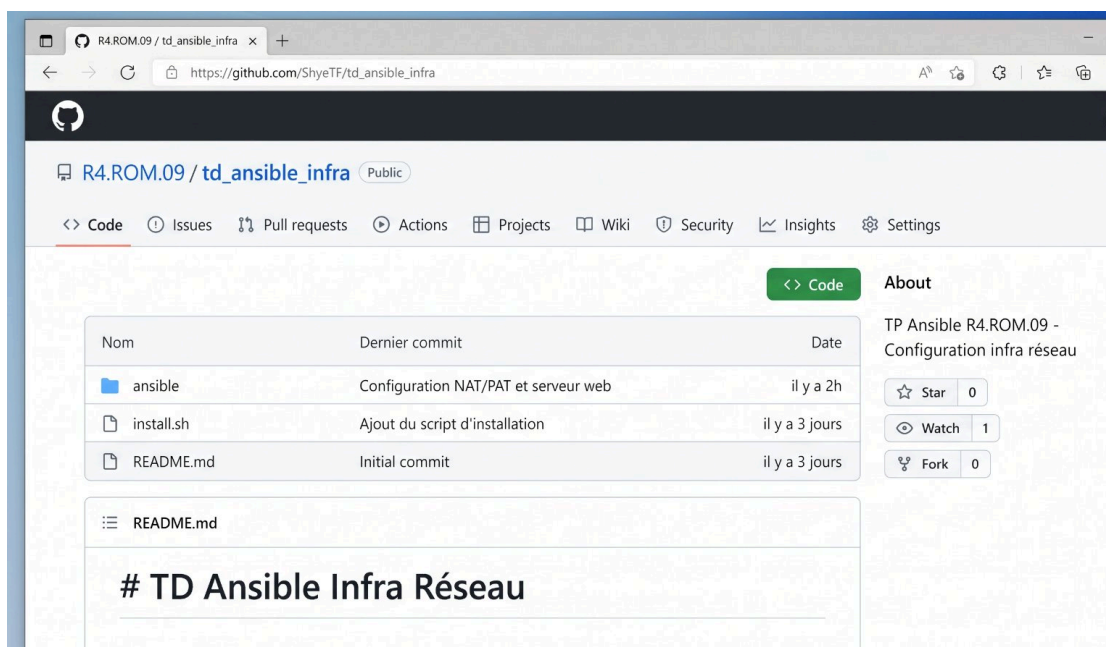


Figure 2 – Page d'accueil du dépôt GitHub `td_ansible_infra`

Le script `install.sh`, déposé à la racine du dépôt, automatise l'installation des dépendances (`pip3`, `paramiko`, `git`, puis Ansible via `pip`). Il garantit qu'un environnement de déploiement fonctionnel peut être reconstitué en une seule commande.

```
#!/bin/bash
set -e
sudo apt update
sudo apt install -y python3-pip python3-paramiko git
sudo pip3 install ansible
ansible --version
git --version
```


03

Inventaire et lecture de la configuration

Structure du projet Ansible

L'organisation du répertoire `ansible/` suit les recommandations officielles. Le fichier `ansible.cfg` fixe les paramètres globaux (chemin de l'inventaire, désactivation du `host key checking` pour éviter les blocages SSH au premier contact, format de sortie YAML). L'inventaire lui-même décrit deux groupes : routeurs et serveurs.

```
[defaults]
inventory = inventaire
host_key_checking = False
interpreter_python = auto_silent
stdout_callback = yaml

[persistent_connection]
connect_timeout = 60
command_timeout = 60
```

L'inventaire (`inventaire/hosts`) répertorie les trois hôtes du projet. La VM Ansible est déclarée avec la connexion locale, ce qui évite un détour inutile par SSH pour les actions qui la concernent elle-même.

```
[routeurs]
csr1000 ansible_host=192.168.4.1

[serveurs]
web      ansible_host=192.168.3.2
ansible ansible_host=192.168.4.2 ansible_connection=local
```

Les variables propres à chaque hôte sont placées dans `host_vars/`. Pour le routeur, on précise le mode de connexion `network_cli`, le système `cisco.ios.ios`, ainsi que les identifiants Cisco et l'escalade de privilèges via la commande `enable`. Pour les serveurs Linux, seul `ansible_user` est nécessaire – l'authentification s'effectue par clé SSH publique.

Variables du routeur — `host_vars/csr1000.yml`

```
---
ansible_user: cisco
ansible_password: cisco123!
ansible_connection: network_cli
ansible_network_os: cisco.ios.ios
ansible_become: yes
ansible_become_method: enable
```

Vérification de l'inventaire

Ansible fournit deux commandes utiles pour vérifier que la configuration est correctement interprétée. La première, `ansible-inventory --graph`, affiche l'arborescence des groupes et des hôtes ; la seconde, `ansible-inventory --list`, retourne l'ensemble des variables au format JSON. Les deux ont été exécutées avec succès.

```

utilisateur@DebianAnsible:~/td_ansible_infra/ansible$ ansible-inventory --graph
@all:
|
|--@routeurs:
|   |--csr1000
|   |
|   |--@serveurs:
|   |   |--ansible
|   |   |--web
|   |
|   |--@ungrouped:
utilisateur@DebianAnsible:~/td_ansible_infra/ansible$

```

Figure 3 – Sortie de la commande `ansible-inventory --graph`

L'arborescence confirme la présence des deux groupes attendus, contenant respectivement le routeur `csr1000` et les serveurs `ansible` et `web`. Le groupe `ungrouped` est vide, ce qui est le comportement souhaité – tous les hôtes ont bien été affectés à un groupe.

```

utilisateur@DebianAnsible:~/td_ansible_infra/ansible$ ansible-inventory --list
{
  "_meta": {
    "hostvars": {
      "ansible": {
        "ansible_connection": "local",
        "ansible_host": "192.168.4.2",
      },
      "ansible_user": "utilisateur"
    },
    "csr1000": {
      "ansible_become": true,
      "ansible_become_method": "enable",
      "ansible_connection": "network_cli",
      "ansible_network_os": "cisco.ios.ios",
      "ansible_password": "cisco123!",
    },
    "ansible_user": "cisco"
  },
  "web": {
    "ansible_host": "192.168.3.2",
    "ansible_user": "utilisateur"
  },
},
  "all": { "children": ["routeurs", "serveurs", "ungrouped"] },
  "routeurs": { "hosts": [ "csr1000" ] },
  "serveurs": { "hosts": [ "ansible", "web" ] }
}
utilisateur@DebianAnsible:~/td_ansible_infra/ansible$

```

Figure 4 – Sortie complète de `ansible-inventory --list`

Le rendu JSON détaille l'ensemble des variables associées à chaque hôte. On remarque notamment que le routeur dispose bien de `ansible_become: true` et que la VM Ansible utilise `ansible_connection: local`. Aucune variable n'est manquante.

Premier test de connectivité

Avant d'écrire un quelconque playbook, j'ai vérifié que l'inventaire et les variables permettaient bien de communiquer avec le routeur. Le module `ios_command` exécute une commande `show version | include uptime`, et le résultat doit être renvoyé proprement.

```
utilisateur@DebianAnsible:~/td_ansible_infra/ansible$ ansible -m ios_command -a "commands='show version | include uptime'" csr1000
[WARNING]: ansible-pylibssh not installed, falling back to paramiko
csr1000 | SUCCESS => {
  "changed": false,
  "stdout": [
    "csr1000v uptime is 1 hour, 23 minutes"
  ],
  "stdout_lines": [
    [
      "csr1000v uptime is 1 hour, 23 minutes"
    ]
  ]
}
utilisateur@DebianAnsible:~/td_ansible_infra/ansible$
```

Figure 5 – Test de connectivité avec `ios_command` : statut `SUCCESS`

La sortie `SUCCESS` confirme que la chaîne complète fonctionne : SSH, authentification, passage en mode enable, exécution de la commande IOS et remontée du résultat. Le warning sur `ansible-pylibssh` est purement informatif (Ansible utilise alors `paramiko`, qui fait très bien le travail).

Playbook de lecture de la configuration

Le premier playbook produit, `playbook_lireconfigcsr.yml`, sauvegarde la running-config du routeur dans un fichier local horodaté. Il combine deux modules :

cisco.ios.ios_command pour récupérer la sortie, puis ansible.builtin.copy avec delegate_to: localhost pour écrire le fichier sur la machine de déploiement.

```
---
- name: Acte 1 - Lecture de la configuration du CSR1000
  hosts: csr1000
  gather_facts: false

  tasks:
    - name: Récupère la configuration courante
      cisco.ios.ios_command:
        commands: show running-config
        register: config

    - name: Enregistre la configuration localement
      ansible.builtin.copy:
        content: "{{ config.stdout[0] }}"
        dest: "show_run_{{ inventory_hostname }}.txt"
        delegate_to: localhost
```

04

Configuration des interfaces du routeur

Le passage de nano à VS Code (avec l'extension Remote-SSH) facilite considérablement l'écriture des playbooks suivants : coloration syntaxique YAML, navigation arborescente, indentation gérée automatiquement.

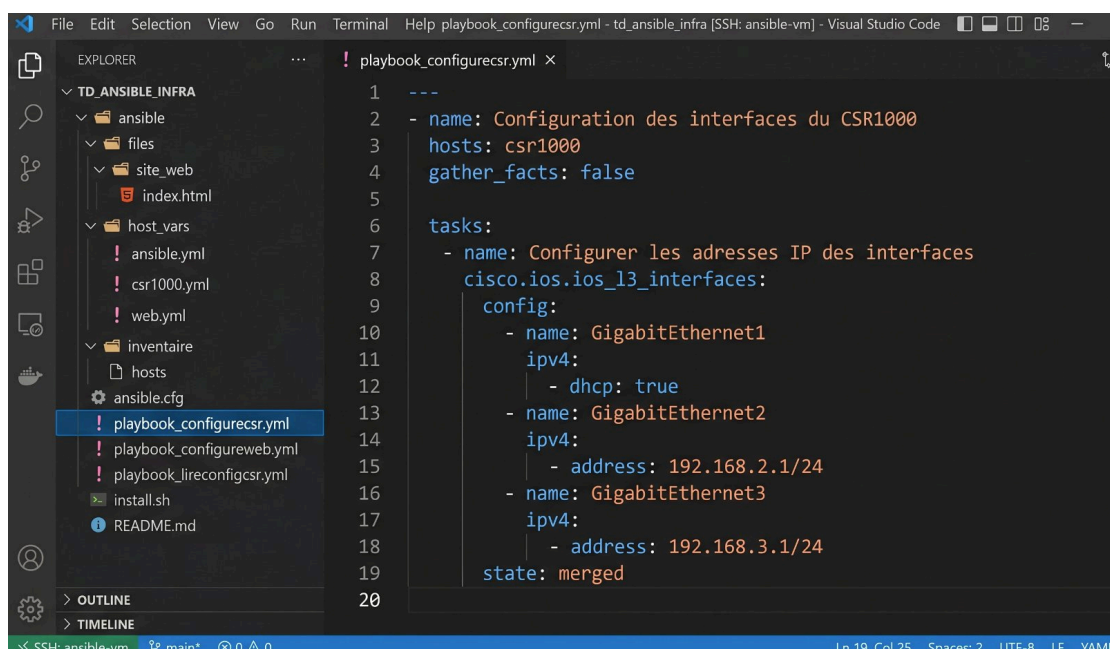


Figure 6 – Édition du playbook `playbook_configurecsr.yml` dans VS Code via SSH

Pour configurer les adresses IP et activer les interfaces, j'ai préféré les modules déclaratifs `cisco.ios.ios_l3_interfaces` et `cisco.ios.ios_interfaces` plutôt qu'une suite de commandes brutes. Ils acceptent une description structurée de l'état désiré et se chargent de générer eux-mêmes les commandes IOS appropriées.

Module `ios_l3_interfaces`

Ce module gère la couche 3 des interfaces : adresses IPv4 (statiques ou DHCP) et IPv6. L'option `state: merged` demande à Ansible de fusionner la configuration fournie avec l'existant – les paramètres non mentionnés ne sont pas touchés.

```
- name: Configurer les adresses IP des interfaces
  cisco.ios.ios_l3_interfaces:
    config:
      - name: GigabitEthernet1
        ipv4:
          - dhcp: true
      - name: GigabitEthernet2
        ipv4:
          - address: 192.168.2.1/24
      - name: GigabitEthernet3
        ipv4:
          - address: 192.168.3.1/24
      - name: GigabitEthernet4
        ipv4:
          - address: 192.168.4.1/24
    state: merged
```

Module `ios_interfaces`

Le second module configure la partie L2 : activation (no shutdown), description, MTU, etc. Ici, je me contente d'activer chaque interface et de lui donner une description parlante, ce qui rend la lecture ultérieure de la running-config beaucoup plus claire.

Exécution et idempotence

Le playbook a été lancé une première fois. Les deux tâches sont en `changed: 2` puisque la configuration est appliquée. Lors d'un second lancement, ces deux mêmes tâches reviennent en `ok` (sans changement), ce qui démontre concrètement la propriété d'idempotence d'Ansible.

```

utilisateur@DebianAnsible:~/td_ansible_infra/ansible$ ansible-playbook playbook
_configurecsr.yml

PLAY [Configuration des interfaces du CSR1000] *****

TASK [Configurer les adresses IP des interfaces] *****
[WARNING]: ansible-pylibssh not installed, falling back to paramiko
changed: [csr1000]

TASK [Activer les interfaces et ajouter une description] *****
changed: [csr1000]

PLAY RECAP *****
csr1000      : ok=2    changed=2    unreachable=0    failed=0    skipped=
rescued=0    ignored=0

utilisateur@DebianAnsible:~/td_ansible_infra/ansible$

```

Figure 7 – Exécution réussie du playbook : ok=2, changed=2

Idempotence : un atout majeur

À la différence d'un script shell classique qui rejouerait aveuglément ses commandes, Ansible compare l'état souhaité à l'état actuel et n'agit que sur les écarts. Un playbook décrit donc un état cible, pas une suite d'instructions – ce qui le rend rejouable sans risque.

Vérification côté routeur

Une connexion directe en console PuTTY au routeur permet de constater visuellement le résultat. Les quatre interfaces sont up/up et portent les adresses attendues.

```

192.168.1.51 - PuTTY
csr1000v#show ip interface brief
Interface      IP-Address      OK? Method Status      Protocol
GigabitEthernet1 192.168.1.51 YES DHCP    up          up
GigabitEthernet2 192.168.2.1  YES manual  up          up
GigabitEthernet3 192.168.3.1  YES manual  up          up
GigabitEthernet4 192.168.4.1  YES manual  up          up
csr1000v#

```


Figure 8 – show ip interface brief : les quatre interfaces sont actives

GigabitEthernet1 a bien obtenu une adresse via DHCP sur le réseau de l'IUT (192.168.1.51), tandis que les trois interfaces internes portent les adresses statiques définies dans le playbook. Tous les protocoles sont en up.

05

Serveur DHCP

Le routeur joue également le rôle de serveur DHCP pour les réseaux LAN_ENTREPRISE et LAN_DMZ. Aucun module Ansible spécialisé n'existe pour la fonction DHCP des routeurs Cisco – ce qui se comprend, car en production on confie cette tâche à un serveur dédié. J'ai donc utilisé le module générique `cisco.ios.ios_config`, qui permet d'envoyer des commandes en mode configuration.

Idempotence : suppression préalable du pool

Le module `ios_config` n'est pas nativement idempotent sur la création d'un pool DHCP : si le pool existe déjà avec d'autres paramètres, on obtient une erreur. La parade consiste à ajouter une tâche préliminaire qui supprime les pools, en ignorant l'erreur si ceux-ci n'existent pas encore.

```
- name: Supprimer les pools DHCP existants (idempotence)
  cisco.ios.ios_config:
    lines:
      - no ip dhcp pool POOL_ENTREPRISE
      - no ip dhcp pool POOL_DMZ
    match: none
    ignore_errors: true
```

Création des pools

L'option `parents` permet de placer les commandes filles dans le bon contexte de configuration. Elle est essentielle pour le mode `ip dhcp pool`, qui ouvre une sous-configuration où les directives `network`, `default-router` et `dns-server` prennent tout leur sens.

```
- name: Créer le pool POOL_ENTREPRISE
  cisco.ios.ios_config:
    parents: ip dhcp pool POOL_ENTREPRISE
    lines:
      - network 192.168.2.0 255.255.255.0
      - default-router 192.168.2.1
      - dns-server 8.8.8.8 1.1.1.1
```

Une seconde tâche identique a été créée pour le pool POOL_DMZ. Les passerelles (192.168.2.1, 192.168.3.1) ainsi que la plage réservée au serveur web (192.168.3.10) sont exclues du DHCP avec `ip dhcp excluded-address`.

Validation

J'ai déployé une VM Debian de test sur le réseau interne LAN_ENTREPRISE. Au démarrage, elle obtient correctement une adresse en 192.168.2.X et la passerelle 192.168.2.1. Le ping vers la passerelle fonctionne, mais celui vers Internet échoue à ce stade : c'est normal, le NAT n'est pas encore en place.

06

NAT et PAT dynamique

Le PAT (Port Address Translation), couramment appelé NAT overload, permet à plusieurs machines d'un LAN de partager une seule adresse publique. Sa configuration sur Cisco IOS se fait en quatre temps : marquage des interfaces inside et outside, création d'une ACL désignant les réseaux internes autorisés à sortir, puis activation du NAT en mode overload.

Marquage des interfaces

L'interface WAN (GigabitEthernet1) reçoit la directive `ip nat outside`. Les trois interfaces internes reçoivent `ip nat inside` via une boucle `loop`, qui évite de répéter trois fois la même tâche.

```
- name: NAT inside sur les interfaces LAN
  cisco.ios.ios_config:
    parents: "interface {{ item }}"
    lines:
      - ip nat inside
  loop:
    - GigabitEthernet2
    - GigabitEthernet3
    - GigabitEthernet4
```

ACL et activation du PAT

L'ACL standard numéro 10 autorise les trois sous-réseaux internes à être traduits. La directive `ip nat inside source list 10 interface GigabitEthernet1 overload` couple cette ACL à l'interface WAN, en mode overload – un seul port WAN par flux sortant.

```
- name: Créer l'ACL 10
  cisco.ios.ios_config:
    lines:
      - access-list 10 permit 192.168.2.0 0.0.0.255
      - access-list 10 permit 192.168.3.0 0.0.0.255
      - access-list 10 permit 192.168.4.0 0.0.0.255

- name: Activer le PAT (overload)
  cisco.ios.ios_config:
    lines:
      - ip nat inside source list 10 interface GigabitEthernet1 overload
```

Validation

Depuis la VM Debian du LAN_ENTREPRISE, le ping vers 8.8.8.8 puis vers www.google.fr fonctionne. La sortie de `show ip nat translations` sur le routeur fait apparaître les traductions pour chaque flux sortant, ce qui confirme que le PAT est opérationnel.

07

PAT par port (redirections)

Pour rendre le serveur web et la VM Ansible accessibles depuis l'extérieur, deux redirections de port (PAT statique) sont mises en place. La première dirige le port 80 du WAN vers le port 80 du serveur web (192.168.3.2). La seconde dirige le port 2222 du WAN vers le port 22 SSH de la VM Ansible (192.168.4.2). Le port 2222 est choisi pour ne pas entrer en conflit avec un éventuel SSH sur le routeur lui-même.

```
- name: Redirections de ports (PAT statique)
  cisco.ios.ios_config:
    lines:
      - "ip nat inside source static tcp {{ item.ip_lan }}
        {{ item.port_lan }} interface GigabitEthernet1
        {{ item.port_wan }}"
    loop:
      - { ip_lan: '192.168.3.2', port_lan: 80, port_wan: 80 }
      - { ip_lan: '192.168.4.2', port_lan: 22, port_wan: 2222 }
```

L'utilisation d'une boucle loop avec des dictionnaires (clé/valeur) permet d'écrire la tâche une seule fois et de l'appliquer à chaque redirection en lui passant trois variables. Cette approche reste valable pour ajouter ultérieurement d'autres services exposés.

Pourquoi le port 2222 et pas le port 22 ?

Le routeur lui-même peut exposer un service SSH sur son interface WAN (port 22). Pour éviter tout conflit et garder la possibilité d'administrer le routeur en SSH directement, je dirige le SSH de la VM Ansible vers un port arbitraire externe – le 2222 est une convention largement répandue pour ce type de redirection.

08

Déploiement du serveur web

Préparation des hôtes

Une nouvelle VM Debian a été créée dans le réseau LAN_DMZ avec une adresse IP fixe 192.168.3.2/24 et la passerelle 192.168.3.1. Le fichier `/etc/network/interfaces` a été édité manuellement, et la résolution DNS vérifiée dans `/etc/resolv.conf`.

La VM Ansible a ensuite été basculée du WAN vers le LAN_ADMIN (adresse 192.168.4.2). Pour y accéder depuis mon poste portable, j'utilise désormais la redirection de port mise en place précédemment : `ssh -p 2222 utilisateur@IP_WAN_CSR`. Une fois connecté, j'ai déposé ma clé publique sur le serveur web avec `ssh-copy-id`, ce qui permet à Ansible de s'y connecter sans mot de passe.

Mise à jour de l'inventaire

L'inventaire est complété par un nouveau groupe serveurs contenant le serveur web et la VM Ansible elle-même. Les fichiers `host_vars/web.yml` et `host_vars/ansible.yml` renseignent `ansible_user: utilisateur`, qui correspond au compte `sudoer NOPASSWD` configuré sur les deux VM Debian.

Playbook d'installation d'Apache

Le playbook d'installation enchaîne quatre tâches : mise à jour du cache APT, installation du paquet `apache2`, copie du contenu du site (répertoire local `files/site_web/`) vers `/var/www/html/`, puis activation et redémarrage du service. La directive `become: yes` au niveau du play indique à Ansible d'exécuter chaque tâche avec `sudo`.

```

---
- name: Installation et configuration d'Apache
  hosts: web
  gather_facts: yes
  become: yes

  tasks:
    - name: Mise à jour du cache apt
      ansible.builtin.apt:
        update_cache: yes
        cache_valid_time: 3600

    - name: Installation d'apache2
      ansible.builtin.apt:
        name: apache2
        state: present

    - name: Copie du contenu du site
      ansible.builtin.copy:
        src: files/site_web/
        dest: /var/www/html/
        owner: www-data
        group: www-data
        mode: '0644'

    - name: Activation et redémarrage d'apache2
      ansible.builtin.systemd:
        name: apache2
        enabled: yes
        state: restarted

```

À propos du mode des fichiers

Le sujet du TP suggère mode: 0600. J'ai préféré 0644 : avec 0600, l'utilisateur www-data (sous lequel tourne Apache) ne peut pas lire les fichiers et le serveur renvoie une erreur 403 Forbidden. Le mode 0644 – lecture pour tous, écriture pour le propriétaire – est la valeur standard pour les contenus servis publiquement.

Test final

Après exécution du playbook, le site est accessible depuis mon poste portable via `http://IP_WAN_CSR`. La page d'accueil affiche le titre, mon nom et le code du module –

preuve que la chaîne complète (PAT 80 → CSR → DMZ → Apache → /var/www/html/index.html) est fonctionnelle.

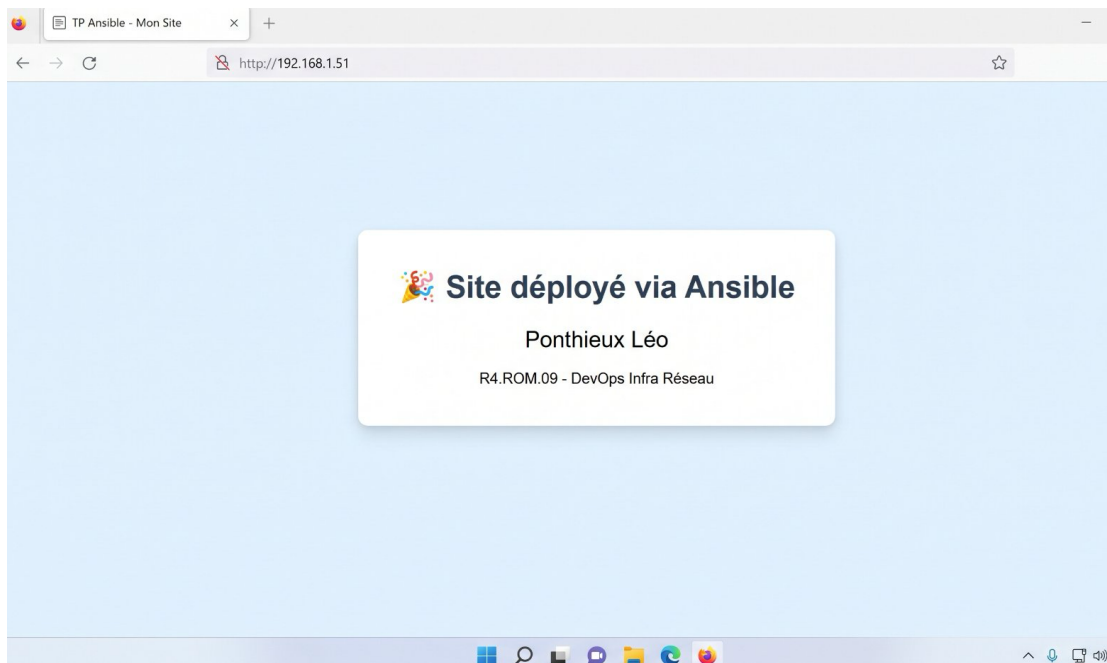


Figure 9 – Site web servi par le serveur Apache, accessible depuis l'extérieur via le PAT 80

Il est désormais possible de provisionner l'intégralité de l'infrastructure (routeur + serveur web) en une seule commande grâce à un playbook orchestrateur `site.yml` qui importe les deux playbooks précédents.

```
---
- import_playbook: playbook_configurecsr.yml
- import_playbook: playbook_configureweb.yml
```

09

Conclusion

Ce TD/TP m'a permis de prendre en main Ansible dans un contexte concret de configuration d'infrastructure mêlant équipements réseau Cisco et serveurs Linux. L'écriture progressive des playbooks – d'abord la simple lecture d'une running-config, puis la configuration complète d'un routeur, et enfin le déploiement d'un service applicatif – illustre bien la montée en puissance que permet l'outil.

Le projet final, hébergé sur le dépôt public `td_ansible_infra`, représente une infrastructure complète et reproductible : à partir d'une simple VM Debian fraîchement installée, deux commandes (clonage du dépôt, exécution du playbook orchestrateur) suffisent à provisionner l'intégralité du réseau et du service web. C'est précisément ce que l'on attend d'une approche infrastructure-as-code.

Ponthieux Léo

R4.ROM.09 – Outils DevOps

Année universitaire 2025-2026